Homework 4: Unsupervised Learning and Expectation Maximization

Liam Youm

Abstract

This report details the implementation of K-Means clustering and Gaussian Mixture Models (GMM) using the Expectation-Maximization (EM) algorithm. These unsupervised learning techniques were applied to the `hw4_dataset.csv` to identify underlying cluster structures. The process of determining the optimal number of clusters, k, is discussed, along with a comparison of the custom implementations with their scikit-learn counterparts. Finally, an interpretation of the clusters in the context of the dataset's presumed animal species is provided.

1 Introduction

Unsupervised learning plays a crucial role in discovering hidden patterns in data where explicit labels are not available. This assignment focuses on implementing two fundamental clustering algorithms: K-Means and Gaussian Mixture Models (GMM) with Expectation-Maximization (EM). The objective was to apply these algorithms to a dataset containing weight (kg) and length (cm) measurements of various animals, identify potential species groupings, and compare the custom implementations with established versions from the scikit-learn library.

2 Methodology: Implemented Algorithms

2.1 K-Means Clustering

K-Means is an iterative algorithm that aims to partition N data points into k distinct, non-overlapping clusters. Each cluster is characterized by its centroid (mean). The algorithm proceeds as follows:

1. **Initialization**: k initial centroids are chosen. My implementation selects k random data points from the dataset as initial centroids.
2. **Assignment Step**: Each data point is assigned to the cluster whose centroid is closest, typically based on Euclidean distance.
3. **Update Step**: The centroid of each cluster is recomputed as the mean of all data points assigned to that cluster.
4. **Convergence**: Steps 2 and 3 are repeated until the centroids no longer change significantly (that is, the sum of squared differences between old and new centroids is less than a tolerance, 'tol'), or a maximum number of iterations ('max_iters') is reached.

My implementation stores the final centroids in 'self.centroids' and the labels for each data point in 'self.labels'.

2.2 Gaussian Mixture Models (GMM) with EM

GMM assumes that the data is generated from a mixture of k multivariate Gaussian distributions, each representing a component (cluster). The Expectation-Maximization (EM) algorithm is used to estimate the parameters of these Gaussian components.

1. **Initialization**: The parameters for each of the k Gaussian components are initialized:
    - Means ($\boldsymbol{\mu}_k$): My implementation initializes means by randomly selecting k data points from the dataset.

- Covariances ($\mathbf{\Sigma}_k$): Initialized based on the overall covariance of the dataset, with a small regularization term ('reg covar', e.g., 10^{-6}) added to the diagonal to ensure numerical stability and prevent singularity. All components start with this same initial covariance structure.
- Mixing Coefficients (π_k): Initialized uniformly as $1/k$ for all components, ensuring $\sum_k \pi_k = 1$.

These learned parameters are stored in 'self.means_', 'self.covariances_', and 'self.weights_' respectively.

2. **E-Step (Expectation Step)**: Given the current parameters, the probability (responsibility) $\gamma(z_{nk})$ that data point x_n belongs to component k is calculated:

$$\gamma(z_{nk}) = \frac{\pi_k \mathcal{N}(x_n|\boldsymbol{\mu}_k, \mathbf{\Sigma}_k)}{\sum_{j=1}^{K} \pi_j \mathcal{N}(x_n|\boldsymbol{\mu}_j, \mathbf{\Sigma}_j)}$$

The log-likelihood of the data given the current parameters is also computed in this step:

$$\ln p(X|\boldsymbol{\mu}, \mathbf{\Sigma}, \boldsymbol{\pi}) = \sum_{n=1}^{N} \ln \left\{ \sum_{k=1}^{K} \pi_k \mathcal{N}(x_n|\boldsymbol{\mu}_k, \mathbf{\Sigma}_k) \right\}$$

My implementation uses 'scipy.stats.multivariate_normal.pdf' for calculating $\mathcal{N}(\cdot)$.

3. **M-Step (Maximization Step)**: The model parameters are re-estimated using the responsibilities calculated in the E-step:

- $N_k = \sum_{n=1}^{N} \gamma(z_{nk})$
- $\boldsymbol{\mu}_k^{\text{new}} = \frac{1}{N_k} \sum_{n=1}^{N} \gamma(z_{nk}) x_n$
- $\mathbf{\Sigma}_k^{\text{new}} = \frac{1}{N_k} \sum_{n=1}^{N} \gamma(z_{nk})(x_n - \boldsymbol{\mu}_k^{\text{new}})(x_n - \boldsymbol{\mu}_k^{\text{new}})^T + \text{reg\_covar} \cdot I$
- $\pi_k^{\text{new}} = \frac{N_k}{N}$

The 'reg_covar' term is added to the diagonal of the re-estimated covariance matrices as well.

4. **Convergence**: Steps 2 and 3 are repeated until the change in log-likelihood between iterations falls below a tolerance ('tol'), or a maximum number of iterations ('max_iters') is reached. My implementation stores the history of log-likelihood values.

3 Determining the Number of Clusters (k)

The choice of k, the number of clusters (or components), is crucial for both K-Means and GMM. I experimented with k values from 2 to 8.

3.1 K-Means: Optimal k Selection

For K-Means, the Elbow method (using Within-Cluster Sum of Squares, WCSS) and Silhouette Analysis were employed.

- **WCSS Results**: The WCSS scores were: k=2: 1,833,832.63; k=3: 1,519,559.49; k=4: 1,413,498.56; k=5: 190,129.46; k=6: 254,625.36; k=7: 148,755.63; k=8: 136,866.33. A plot of WCSS vs. k showed a dramatic drop from $k = 4$ to $k = 5$. The WCSS for $k = 6$ was anomalously higher than for $k = 5$, likely due to initialization sensitivity of K-Means leading to a less optimal local minimum for that specific run with $k = 6$. The most significant "elbow" appeared at $k = 5$.
- **Silhouette Scores**: The average Silhouette scores were: k=2: 0.8569; k=3: 0.7121; k=4: 0.7957; k=5: 0.7950; k=6: 0.5918; k=7: 0.6480; k=8: 0.6979. The highest score was for $k = 2$. However, $k = 5$ also yielded a high score, comparable to $k = 4$.

Justification for $k = 5$ (K-Means): Considering the significant decrease in WCSS at $k = 5$ (suggesting a major improvement in compactness) and a strong Silhouette score, $k = 5$ was chosen as the optimal number of clusters for K-Means. While $k = 2$ had the highest Silhouette score, visual inspection of the data suggested more than two distinct groupings. The anomalous WCSS for $k = 6$ was attributed to initialization randomness.

3.2 GMM: Optimal Number of Components (k) Selection

For GMM, the number of components k was evaluated using the trend of Log-Likelihood vs. k and Silhouette Analysis. Models from $k = 2$ to $k = 6$ converged within 150 iterations, while $k = 7$ and $k = 8$ did not.

- **Log-Likelihood Results (for converged models)**: k=2: -9232.88; k=3: -8302.06; k=4: -7988.05; k=5: -7625.83; k=6: -7616.53. The log-likelihood generally increases with k. The increase from $k = 4$ to $k = 5$ was substantial (approx. 362), while the increase from $k = 5$ to $k = 6$ was marginal (approx. 9). This suggests diminishing returns in model fit beyond $k = 5$ for converged models.
- **Silhouette Scores (for converged models)**: k=2: 0.8569; k=3: 0.6882; k=4: 0.6874; k=5: 0.7950; k=6: 0.6621. For converged models, $k = 2$ had the highest Silhouette score, followed by $k = 5$.

Justification for $k = 5$ (GMM): Considering the significant gain in log-likelihood up to $k = 5$ and its stabilization thereafter (minimal gain for $k = 6$), combined with a high Silhouette score (second only to $k = 2$ among converged models), $k = 5$ was chosen as the optimal number of components for GMM. This choice also aligns with the number of clusters selected for K-Means, allowing for a more direct comparison of the two algorithms' ability to model the data with the same number of underlying groups. Visual inspection of preliminary GMM plots for $k = 5$ also suggested a reasonable partitioning of the data.

4 Clustering Results and Visualizations (k=5)

4.1 K-Means Clustering Result

My K-Means implementation was run with $k = 5$. Figure 1 visualizes the resulting cluster assignments and centroid locations.

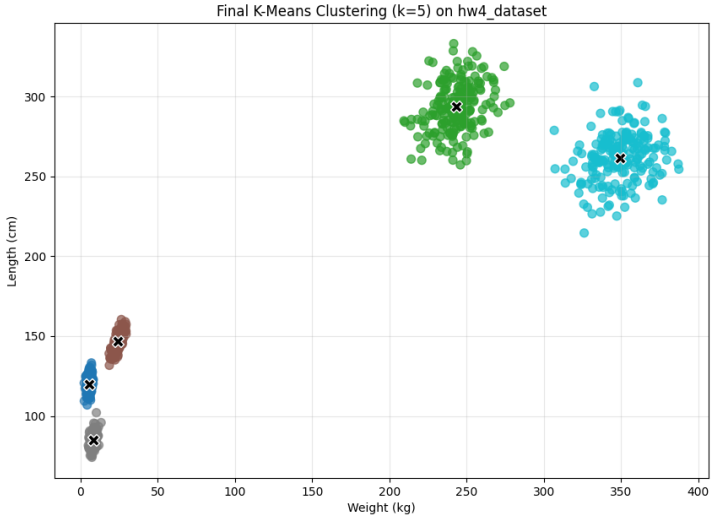

Figure 1: My K-Means Clustering Result with $k = 5$.

The clusters appear to partition the data well, including small clusters at the bottom-left corner.

4.2 GMM Clustering Result

My GMM implementation was run with $k = 5$ components. The resulting cluster assignments (based on highest responsibility) and component means are visualized in Figure 2.

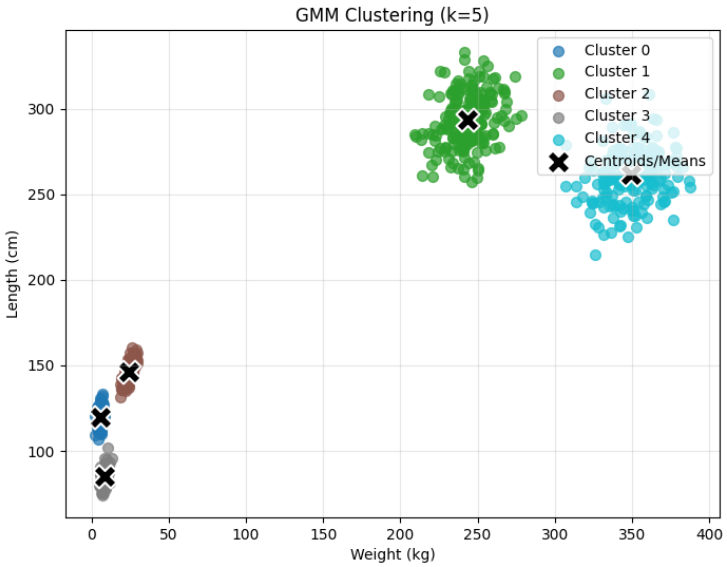

Figure 2: My GMM Clustering Result with $k = 5$ components.

The GMM, with its ability to model covariances, might capture the shapes of the underlying distributions more flexibly than K-Means, even if ellipses are not explicitly drawn here.

5 Comparison with Scikit-learn (Brief Overview)

Both my K-Means and GMM implementations (with $k = 5$) were compared to their counterparts in scikit-learn.

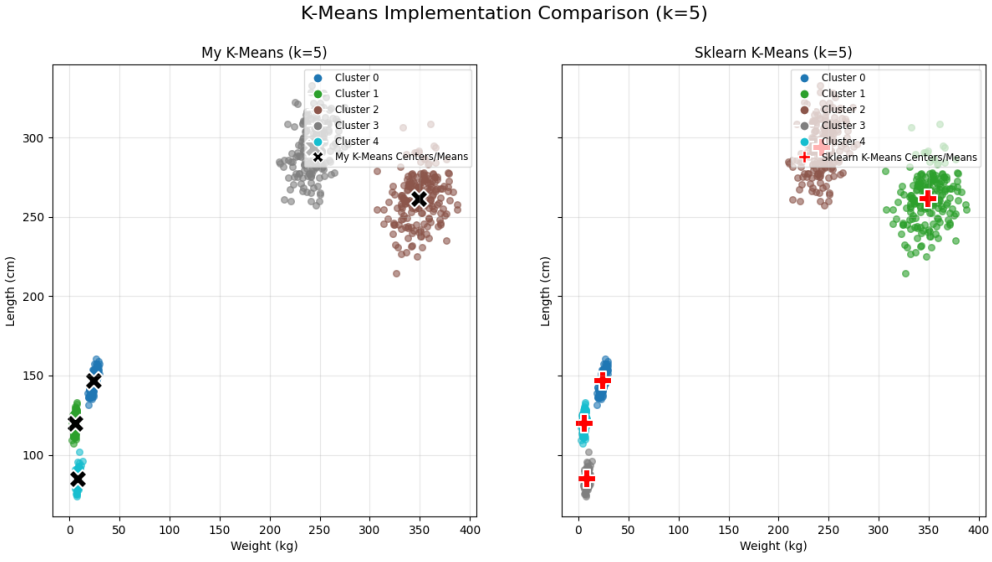

Figure 3: K-means Comparison with Scikit-learn

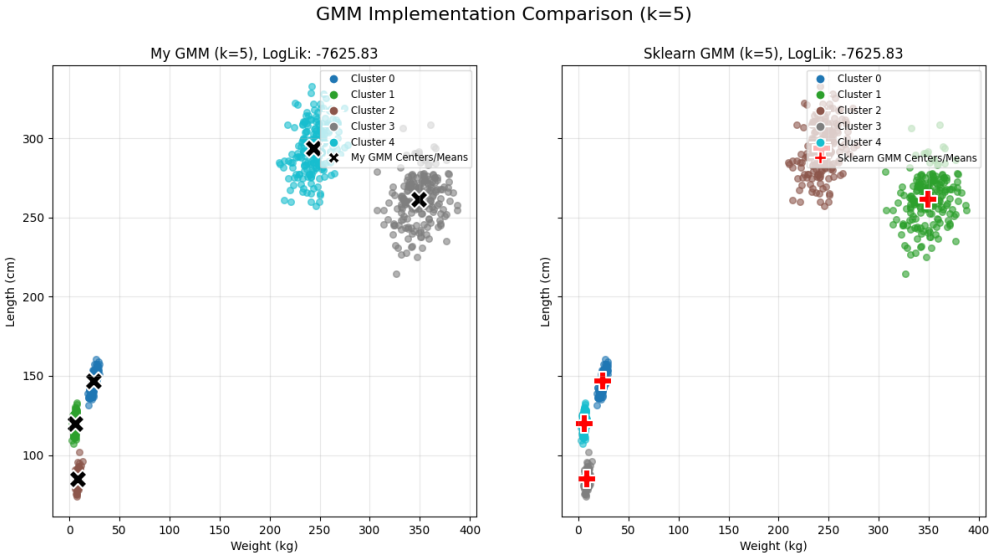

Figure 4: GMM Comparison with Scikit-learn

The two comparison results above show the best outcomes from my algorithm compared with the model implemented in scikit-learn. As shown, similar clustering can be achieved when the best results are obtained, but the same results are not always guaranteed.

Potential reasons for differences include:

- **Initialization**: Scikit-learn's 'KMeans' uses 'k-means++' initialization and 'n_init ¿ 1' by default, making it more robust to poor initializations. My K-Means used a single random initialization. Scikit-learn's 'GaussianMixture' often uses k-means for 'init_params' and also has 'n_init'. My GMM used a random initialization scheme.
- **Algorithmic Optimizations**: Scikit-learn implementations are highly optimized and may include minor algorithmic variations for speed or numerical stability.
- **Convergence Criteria**: While 'tol' and 'max_iters' were set similarly, the exact point of convergence or internal checks might differ.
- **Local Optima**: Both K-Means and EM for GMM can converge to local optima. Different initializations and minor implementation details can lead to different local optima being found.

Despite these differences, my implementations demonstrated the core principles of the algorithms and produced reasonable clusterings on the dataset.

6 Animal Speculation

Based on the $k = 5$ clustering (using GMM results as an example, or K-Means if preferred), the clusters can be characterized by their mean weight (kg) and length (cm).

Based on these values, here are some speculations, keeping in mind that these are rough estimates and animals were found by UC Santa Cruz students (suggesting local California fauna or commonly studied animals):

- **Cluster with low weight, low length**: Could represent small rodents (e.g., mice, voles), small birds, or perhaps reptiles/amphibians like lizards or salamanders.
- **Cluster(s) with moderate weight, moderate length**: Could be medium-sized mammals like raccoons, skunks, rabbits, or larger birds like hawks or owls.
- **Cluster with higher weight, higher length**: Could represent larger mammals such as deer, coyotes, or possibly larger marine animals if the "world" a biology major explores includes coastal areas (though less likely for simple length/weight without more context).

- **Elongated clusters (if GMM showed this)**: Might suggest animals with a significant length-to-weight ratio difference, like snakes or weasels, if one dimension is consistently larger relative to the other for that group.

Further biological knowledge or more features would be needed for accurate identification. This speculation is based purely on the two provided dimensions.

7 Conclusion

This assignment involved the successful implementation of K-Means and GMM clustering algorithms from scratch. The process of selecting an appropriate number of clusters, $k = 5$, was explored using methods like WCSS, Silhouette analysis, and log-likelihood trends, complemented by visual inspection. The custom implementations produced reasonable clusterings on the 'hw4_dataset.csv' and provided a basis for comparison with scikit-learn's versions, highlighting the impact of initialization and algorithmic details. The resulting clusters allowed for tentative speculation on the types of animals measured.